EE5203 L09 Study Sheet

ADC and sensor acquisition - Basri Erdogan

Essential question: Every signal so far has been high or low. A potentiometer, a light sensor, a temperature — these are neither. How does a voltage become a number your program can reason about, and how much should you trust it?

What you should be able to do

- Explain resolution, LSB size, and the counts millivolts conversion.
- Configure ADC1 and justify the resolution, sampling time, and mode.
- Read a channel by polling, paced by the L07 tick, and switch channels safely.
- Map counts to a control value in integer arithmetic.
- Verify `MODER`, `DR`, `SMPR2` and `SQR3` in the SFR view.

Counts and millivolts

$$\text{mV} = \frac{\text{raw} \times 3300}{4095} \qquad \text{LSB} = \frac{3300\ \text{mV}}{4095} \approx 0.806\ \text{mV}$$

`raw`	mV	LED duty (`×1000/4095`)	Servo (`500 + ×2000/4095`)
0	0	0	500 µs — 0°
1024	825	250	1000 µs — 45°
2048	1650	500	1500 µs — 90°
3072	2475	750	2000 µs — 135°
4095	3300	1000	2500 µs — 180°

The duty scale ends at **1000**, not 999: with `ARR` = 999, `CCR1` = 1000 is the real 100 % (the +1 rule of L08).

- **12 bits** means 4096 levels, `0` … `4095`. Full scale (4095) maps to full reference (3300 mV) — the endpoint convention.
- **Integers throughout.** `(raw * 3300u) / 4095u` fits comfortably in `uint32_t`. Multiply *before* dividing, or you lose everything to truncation. Convert to float only at a display edge, if at all.
- **The reference is the ruler.** Every count is relative to VREF. If the 3.3 V rail sags, every reading is wrong in the same direction, and nothing in the data shows it.

Vocabulary table

Name	Job
Resolution	how many levels — 12 bits → 4096
LSB	the voltage one count represents 0.806 mV
Sampling time	how long the input charges the internal hold capacitor
Conversion time	sampling + 12 SAR cycles
`EOC` (`SR`)	end of conversion — a result is waiting; `HAL_ADC_PollForConversion()` waits for it and clears it
`DR`	data register — holds the last result until the next conversion ends
`CONT`	continuous mode: convert again immediately, forever
SAR	successive approximation — the binary-search guessing game

Configuration and reading

```
/* CubeMX: PA0 -> ADC1_IN0, PA1 -> ADC1_IN1, 12 bits,
   continuous conversion Disabled, sampling time 84 Cycles. */

uint32_t adc_read(uint32_t channel)
{
    ADC_ChannelConfTypeDef s = {0};
    s.Channel      = channel;
    s.Rank         = 1;
    s.SamplingTime = ADC_SAMPLETIME_84CYCLES;
```

```
    HAL_ADC_ConfigChannel(&hadc1, &s);       /* select the channel */

    HAL_ADC_Start(&hadc1);                   /* start one conversion */
    HAL_ADC_PollForConversion(&hadc1, 2);    /* wait for EOC, about 12 us */
    return HAL_ADC_GetValue(&hadc1);         /* read DR */
}
```

Pace the reads from the tick, never from `HAL_Delay`:

```
if ((g_ms - t_adc) >= 10U) {            /* L07 tick schedules the read */
    t_adc = g_ms;
    uint32_t raw  = adc_read(ADC_CHANNEL_0);
    uint32_t duty = (raw * 1000u) / 4095u;            /* 0 ... 1000 = ARR + 1 */
    TIM2->CCR1 = duty;
}
```

Conversion time = (sampling time + 12 cycles) ÷ ADC clock. The ADC clock is 16 MHz ÷ 2 = 8 MHz, so 84 cycles give (84 + 12) ÷ 8 MHz = **12 µs**; the CubeMX default of 3 cycles gives 1.9 µs.

Sampling time. The input must charge the sample-and-hold capacitor through the source's own resistance. A low-impedance source (a potentiometer wiper) charges fast; a high-impedance one (an LDR divider, a thermistor) needs longer. Too short and the reading is dragged toward the *previous* channel's value — the classic symptom of cross-talk when switching channels.

One ADC, many channels. There is one converter behind a multiplexer. To read two channels you reconfigure rank 1 and convert again — which is what `adc_read(ch)` does. Both readings are therefore taken at slightly different moments, not simultaneously.

The pin goes analog

`MODER` for PA0 reads `11` — analog, the fourth row of the L04 table. In analog mode the digital input is disconnected: the ADC sees the pin voltage, and `HAL_GPIO_ReadPin` on that pin returns 0. An analog pin must stay between 0 V and 3.3 V; it is not 5 V tolerant.

The three flag families, so far

Flag	Peripheral	Set when
`UIF`	timer (L07)	the counter wrapped at `ARR`
`CC1IF`	timer channel (L08)	an edge was captured
`EOC`	ADC (L09)	a conversion finished; cleared by reading `DR`, or by the HAL in `PollForConversion`

Same pattern every time: hardware raises a flag, software tests it or takes an interrupt, and something clears it.

SFR evidence

Claim	Evidence
the pin is analog	`GPIOA->MODER` bits 1:0 = `11`
the result is real	pause after a reading: `ADC1->DR` holds the same number as `raw`, and both follow the potentiometer
the sampling time is 84 cycles	`SMP0` in `ADC1->SMPR2` = `100`
the right channel was converted	`SQ1` in `ADC1->SQR3` = the channel number (1 after the LDR reading)
one conversion for each start	`CONT` in `ADC1->CR2` = 0

`EOC` is not on this list: `HAL_ADC_PollForConversion()` clears it before you can pause, so it reads 0 in the SFR view.

Common mistakes

- Dividing before multiplying: `(raw / 4095) * 3300` is always 0.

- Assuming a 5 V reference and 1023 (Arduino habit) — this board uses 3.3 V and 4095.
- Leaving the sampling time at the CubeMX default of 3 cycles.
- Sampling time too short for a high-impedance sensor; readings drift toward the previously converted channel.
- Reading `DR` without checking `EOC`, and getting the previous conversion.
- Calling `HAL_ADC_Start` once and expecting fresh values forever in single mode.
- Wiring the potentiometer's wiper to a rail instead of the middle terminal.
- Expecting an LDR divider to be linear in light — it is not, and it does not need to be for a threshold.

Mastery self-check

☐ I can convert counts to millivolts and back, in integers.
☐ I can state the LSB size and what resolution buys me.
☐ I can justify a sampling time from the source impedance.
☐ I can explain why two channels are not sampled simultaneously.
☐ I can calculate the conversion time from the sampling time and the ADC clock.
☐ I can predict `MODER` for an analog pin.
☐ I can map a reading onto a duty and a servo pulse without floats.
☐ I can say what happens to every reading if VREF sags.

Five review questions

1. A reading of `2867` arrives on the 12-bit scale. Give the voltage in millivolts, the LED duty it commands via `×1000/4095`, and the servo pulse via `500 + ×2000/4095`.
2. A student writes `mV = (raw / 4095) * 3300` and always gets 0 (or 3300). Explain precisely what integer division did, and give the correct expression.
3. The potentiometer reads correctly, but the LDR channel returns values that look like a faded copy of the potentiometer's. Name the most likely cause and the one setting you would change.
4. The sampling time is left at 3 cycles. How long is one conversion at an 8 MHz ADC clock, and which symptom does the short sampling time cause when two channels are read one after the other?
5. Your night-light switches LD2 on and off rapidly whenever the room light sits near the threshold. Name the phenomenon, and give a fix that does not involve changing the sensor.

See the L09 worksheet for extended practice. Worked solutions are released after the practice window.